

Code review conversation

Project: saas (Spring Boot 4.1 / Java 21) · Branch: master · Date: 2026-10-07

USER

/code-review review all code.

CLAUDE

A background code-review agent ran and reported 10 findings.

Review 1: correctness and security findings

The review found 10 issues, ranked most severe first. No code was changed.

#	Location	Finding
1	OidcService.java:44	Google login trusts <code>profile.email()</code> with no <code>email_verified</code> or <code>enabled</code> check and links to any existing account by email. An attacker with an unverified Google address matching a victim's email gets tokens for the victim's account (account takeover). Disabled users also get tokens through this path.
2	UserService.java:52	Email normalization is inconsistent. <code>createUser</code> trims and lowercases, but <code>readOrCreateUser</code> , login and forgot-password lookups, and <code>updateUserEmail</code> use the raw email. A user who registers as "Bob@x.com" cannot log in with that address.
3	db/changelog/v1/init.yaml	<code>app_user.email</code> has no unique constraint. Concurrent signups can both insert, after which <code>findByEmail</code> throws and login for that email returns 500 permanently.
4	SecurityUtils.java:16	<code>hash(null)</code> throws an NPE and is called with the null password of OIDC-created users. <code>/forgot-password</code> for a Google account returns 500, which also reveals that the account exists.
5	SecurityService.java:67	The password-reset JWT is written to the application log (no mail delivery), and <code>resetPassword</code> never checks <code>enabled</code> . Anyone with log access can take over any account.
6	UserService.java:87	Changing the password or email does not revoke the user's refresh tokens. Only the reset-password flow does.
7	OidcService.java:37	The OIDC callback holds one DB transaction across two blocking Google calls, with no HTTP timeouts. State is deleted only afterwards, so two concurrent callbacks can both pass validation (not truly single-use). A null email causes an NPE.
8	SecurityService.java:94	The refresh-token reuse grace period is ineffective. A token revoked within the window still throws <code>InvalidTokenException</code> and issues nothing.
9	CreateUserRequest.java:6	Passwords have only <code>@NotBlank</code> , with no minimum or maximum length. BCrypt silently truncates input beyond 72 bytes.
10	PlatformConfiguration.java:9	Annotated <code>@Configuration</code> , which registers a second unbound bean with null fields. <code>PlatformConfigurations</code> is also <code>@Configuration</code> . Separately, <code>.env</code> is tracked in git and not ignored, so real secrets may be committed.

Claude offered to fix these, starting with the top three, and advised rotating any credentials in `.env` since they are already in git history.

USER

/thermo-nuclear-code-quality-review

CLAUDE

Checked git status (clean apart from the untracked `skill` directory, so no branch diff), listed all source files (about 1.6k lines), and read the service, OIDC and security code.

Review 2: code quality (whole codebase, no branch diff)

No structural regressions were found, and the package-by-feature layout is clean. The biggest problems are duplicated code and a few abstractions that cost more than they save. Verdict: would not pass the approval bar yet, mainly because of items 1 to 3.

1. Missed simplification: two near-identical JWT services

`AccessTokenService` and `PasswordResetTokenService` repeat the `@Value` secret and expiration fields, `@PostConstruct` key setup, `createJwt` timing logic, `readJwt`, `readSubject`, and the `JwtException` to `InvalidTokenException` wrapper. Only the claim differs (`role` vs `pwh`).

- **Proposed:** extract one `JwtCodec` (secret, expiration, claims map); each service shrinks to its claim and validity rule. About 60 lines disappear.
- `AccessTokenService.isValidJwt` and `readRole` are likely unused, since the filter does its own reads.
- Tokens are parsed and verified two or three times per request because each accessor calls `readJwt`. Return `Claims` once.
- Snake_case locals: `now_plus_expiration`, `state_`.

2. Misplaced logic: `SecurityService.refresh` and `RefreshTokenService`

- `refresh` is a 35-line chain of guard clauses. The `user.isEmpty()` and `!enabled` branches are identical, so collapse them into `user.filter(UserDto::enabled).isEmpty()`.
- The predicates `isExpiredToken`, `isRevokedToken` and `isWithinReuseGracePeriod` only read DTO fields. Move them onto `RefreshTokenDto` as `isExpired(now)`, `isRevoked()`, `isInGracePeriod(now, grace)`.
- `RefreshTokenService` has two `createToken` and two `revokeTokenFamily` overloads, and `readToken` and `readOptionalToken` overlap. Pick one entry point per concept.
- `revokeToken(UUID)` is a read-modify-save while the other revocations are bulk queries. Make it a query for consistency and atomicity.
- The grace period is dead behavior: it suppresses family revocation but still throws. Implement real grace behavior or delete the setting and branch.

3. Wrong layer and ownership: `OidcService`

- **Interface fan-out:** `UrlBuilder`, `TokenExchanger` and `ProfileFetcher` are three interfaces only `GoogleService` implements, resolved through a generic `resolve(...)` helper. Replace them with one `OidcProvider` injected as `Map<PlatformEnum, OidcProvider>`. This deletes `PlatformScoped`, `resolve`, and two interfaces.

- **Transactions:** class-level `@Transactional` wraps the Google HTTP calls in a DB transaction. `handleCallback` should be non-transactional and delegate DB work to the already-transactional services.
- **Duplicated token issuance:** `createJwt` plus `createToken` appears in `login`, `refresh` and the OIDC callback. Add one `issueTokens(UserDto)`.
- **State handling:** move `validateState` into `StateService.consume(value)` (read, check expiry, delete atomically). This also fixes the single-use race.

4. Boundary and type problems

- `PlatformConfiguration` and `PlatformConfigurations` should be `@ConfigurationProperties` holders (records or plain beans), not `@Configuration`.
- `GoogleService` calls `getConfiguration()` about nine times. Read it once.
- Wildcard import `dto.*` sits alongside explicit imports of the same classes. The relationship between `AccessToken` and `GoogleTokenDto` is implicit.
- `JsonUtils.MAPPER` is a public mutable static. Make it `final`, or remove it if Spring's mapper can be injected.
- `StateService` creates its own `SecureRandom` although `SecurityUtils.SECURE_RANDOM` exists. "32 random bytes to base64" is repeated in three services. Add `SecurityUtils.randomToken()`.
- The two `updateUserPassword` overloads mean different things (reset vs change). Rename to `resetPassword` and `changePassword`.
- `createUser(email, null)` for OIDC users sends a null into `passwordEncoder.encode`, the root of the NPE chain. Model it explicitly with `createPasswordlessUser(email)`.
- Email normalization belongs in one private `normalize(email)` called by every entry point.

5. Nits

- `ResponseEntity.ok(...)` everywhere in the controllers could be a plain DTO return with `@ResponseStatus`.
- Routes are split between `shared/rest/Routes` and per-feature classes, so there are two places to look.

File size

All files are under 130 lines, so there are no decomposition concerns.

Suggested order

1. Extract `JwtCodec` and `issueTokens`.
2. Collapse the OIDC interfaces into one provider and make `StateService.consume` atomic.
3. Move token predicates onto `RefreshTokenDto` and simplify `refresh`.
4. Fix the config classes and the email and password handling.

Expected result: roughly 150 to 200 lines removed with no behavior change. The bug-level issues from Review 1 touch the same code and could be fixed in the same pass, running `./mvnw test` and `spotless:apply` after each step. Claude asked whether to start the refactor with the bug fixes folded in.

USER

/anthropic-skills:pdf can you save this conversation as pdf ?

CLAUDE

This document. It is a cleaned-up transcript: system notifications and raw tool output were left out, and the two reviews are reproduced in full.